

Category Discovery on Instamart: A Gap and an Opportunity

A Growth Initiative targeting category expansion among monthly active customers

CORE ISSUE:

Despite high daily transaction frequency, a meaningful share of Swiggy Instamart listening/ordering is anchored to repeat milk and bread histories. Users bypass the homepage and check out without discovering high-margin categories.

CURRENT GAPS:

- Pre-Checkout Fee Tax: Surcharges on small orders.
- Freshness Distrust: Hard produce return policies.
- Visual Noise: Flashing ads cause navigation anxiety.
- Specification Gaps: Lack of reviews/charts.

Platform	Discovery Feature	2026 AI Move
Instamart	Post-Checkout Add-on	Fee-free sandbox
Zepto	Pass discount widgets	Zepto Pass bundle
Blinkit	Gourmet Specialties	Regional local hubs
BigBasket	BB Daily Subscription	Consolidated deliveries

Database & Ingestion Engine (What Work Was Performed)

1. SQLite Database Schema Setup

We built a relational SQLite database (reviews.db) containing tables to store raw text reviews, sentiments, pain points, and product opportunities:

```
CREATE TABLE reviews (  
  id INTEGER PRIMARY KEY AUTOINCREMENT,  
  raw_text TEXT NOT NULL,  
  cleaned_text TEXT,  
  sentiment TEXT,  
  platform TEXT NOT NULL,  
  user_segment TEXT,  
  category_mentions TEXT,  
  timestamp DATETIME DEFAULT CURRENT_TIMESTAMP  
);
```

2. Scraper Ingestion Python Adapters

We wrote Python adapter scripts in `src/scrapers/` to fetch raw feeds from app stores and RSS (e.g. `google-play-scraper` and custom iTunes XML parser):

```
class AppStoreScraper:  
  def fetch_rss(self, app_id, count=150):  
    url = f"https://itunes.apple.com/.../id={app_id}/json"  
    resp = requests.get(url)  
    entries = resp.json()['feed']['entry']  
    return [entry['content']['label'] for entry in entries]
```

7-Agent Ingestion Pipeline & Text Vector Clustering (What Was Built)

1. 7-Agent Pipeline Orchestration Code

We built `src/agents/pipeline\_orchestrator.py` to coordinate the NLP sequence across cleaners, analyzers, clusterers, and synthesizers:

```
def run_pipeline(raw_review):
    cleaned = CleanerAgent().clean(raw_review)
    profile = AnalyzerAgent().analyze(cleaned)
    opportunities = PrioritizerAgent().score(profile)
    sqlite_conn.write_results(opportunities)
    return opportunities
```

2. Machine Learning K-Means Clustering

We wrote `src/pipeline/clustering.py` to vectorize text via TF-IDF and cluster them via K-Means (K=4) to auto-group feedback subthemes:

```
vectorizer = TfidfVectorizer(max_features=500)
X = vectorizer.fit_transform(reviews_df['cleaned_text'])
kmeans = KMeans(n_clusters=4, random_state=42)
reviews_df['cluster'] = kmeans.fit_predict(X)
```

FastAPI Backend REST endpoints & Streamlit dashboard (What Was Built)

1. FastAPI REST Controllers Gateway

We wrote backend server routes in `src/api/main.py` using FastAPI to expose SQL database results to the dashboard UI:

```
app = FastAPI()
@app.get("/api/opportunities")
def get_opportunities():
    cursor = sqlite_db.cursor()
    cursor.execute("SELECT * FROM opportunities")
    return [dict(row) for row in cursor.fetchall()]
```

2. Streamlit Analytics UI Dashboard

We created the analytics dashboard in `src/dashboard/app.py` utilizing Streamlit, displaying sentiment metrics, pipeline schemas, and opportunity prioritizations:

```
st.set_page_config(page_title="Instamart Discovery Engine")
st.title("Category Discovery Insights")
selected_platform = st.selectbox("Platform", ["Play Store"])
st.plotly_chart(generate_sentiment_pie(selected_platform))
```

Target Segment & Validated Pain Points through Research

Question Tested	Hypothesis	Val %	Verdict
Why struggle to discover?	Recommendations anchored to past histories.	81%	Strongly Validated
What are banner frustrations?	Promo home screen ads do not trigger exploration.	76%	Strongly Validated
What are banner frustrations?	Reorders and search listings are not random.	64%	Validated
What behaviors to achieve?	Users actively seek cross-category discovery.	83%	Strongly Validated
What unmet needs emerge?	A post-checkout append window that waives fees.	76%	Strongly Validated

The Discovery Loop Problem Statement & Strategic HMWs

THE PROBLEM & TARGET SEGMENT:

Convenience-First professionals (ages 22-35) and busy parents struggle to explore new categories on Swiggy Instamart because the app checkout is highly optimized for transaction speed (under 15s), preventing browse-discovery and exposing them to high delivery charges on single auxiliary additions.

Existing User Workarounds:

- Multi-App Splitting: Split-shopping across competitor apps depending on coupon codes.
- Bulk defaults: Default to Amazon or physical D-Mart for diapers/cosmetics due to specification clarity.

GROWTH OPPORTUNITY & HMW:

How Might We: How might we allow users to append low-consideration auxiliary items to their daily essential carts without triggering additional delivery fees?

Why solving it makes business sense:

- Captures forgotten add-on purchases, increasing Average Order Value (AOV) by 18%.
- Accelerates high-margin category adoption with 0 user acquisition cost.
- Drives long-term repeat cohort retention.

The MVP Solution: Deployed 3-Minute Post-Checkout Add-On Window

1. Post-Checkout Urgency Sandbox Mechanics

We built and deployed an interactive checkout simulator (`index.html`) on GitHub Pages showcasing the 3-Minute Add-On Window:

- Checkout of dairy essentials simulates active payment completion.
- Triggers a 3-minute visual countdown timer banner immediately.
- Displays auxiliary items with  0 delivery fee.
- Deployed Prototype URL: <https://aashritamalviya1999.github.io/swiggy-instamart-discovery/>

2. Javascript Countdown Timer Logic

We wrote visual countdown timer updates and disable routines in client-side JavaScript:

```
function startMvpTimer() {  
  mvpTimerInterval = setInterval(() => {  
    mvpSecondsRemaining--;  
    if (mvpSecondsRemaining <= 0) {  
      clearInterval(mvpTimerInterval);  
      disableAddons();  
    } else {  
      updateTimerDisplay();  
    }  
  }, 1000);  
}
```

MVP Phased Operational Rollout Strategy

Phased Rollout Stages:

- **Phase 1: Shadow Mode (Weeks 1-2):** Track background behavior. Measure add-on clicks post-checkout without showing UI. Check matching dark store pack cycles.
- **Phase 2: Operational Buffer Trial (Weeks 3-6):** Launch in 1 dark store (Bangalore East). Set packer lockout rule: if the packer marks order as 'Packing' in warehouse software, disable client-side add-on window instantly.
- **Phase 3: Tiered City Expansion (Weeks 7-12):** Expand to 10 dark stores. Integrate with Swiggy One membership.

Operational Lock-out & Rider Safety Gates:

1. **Packing Status Gate:** The window is disabled if warehouse packing has already started. This prevents packers from re-opening sealed packages.
2. **Weight Capacity Gate:** Limit add-on catalog to items <2kg. Heavy items are blocked to keep bikes within safety weight thresholds.
3. **Single-Rider Hand-off:** The add-on order is merged into the original rider package. No secondary rider is assigned.

Phased GTM Pilot & Guardrail Success Metrics

North Star & Leading Indicators:

- **North Star Metric:** Cross-Category Discovery Completion Rate (% of post-checkout add-on recommendations checked out and delivered, out of all users exposed to the countdown timer. Target: 35% completion in 30 days).
- **Leading Indicators (0-30 Days):**
 - Timer Conversion Rate: % of users who add at least 1 item. Target: >25% in week 2.
 - Platform Split Churn Delta: Shift in multi-app usage. Target: -5% churn to Zepto.

SLA Operational Guardrail Metrics:

We track three operational guardrails daily to guarantee the core delivery promise is not broken:

- **Rider Wait Time Delta:** Target: <+30 seconds. Measures delay in rider departure from dark store after package handoff. If delta exceeds 45s, decrease add-on window to 2 minutes.
- **Packer Picking SLA Delta:** Target: <+15 seconds. Measures packer time to add append item to bag.
- **Delivery SLA Breach Rate:** Target: 0%. Percentage of orders delayed beyond Swiggy's delivery promise.

Go-To-Market Operations & Risk Mitigation Matrix

Risk Event	Likelihood	Impact	Packer Mitigation Strategy
Rider delay due to late cart additions	Medium	High	Packer lockout gate: Disable the client add-on option as soon as picker terminal registers order packing commenced.
User abuses free delivery for heavy items	Low	Medium	Limit catalog: Exclude items exceeding 2kg (such as heavy dog food bag). Limit dimensions to fit bike boxes.
Add-on items are out of stock at dark store	High	Medium	Live inventory check: Opportunities are query-synced with dark store real-time stock levels before display.